University of Central Punjab

FOIT (Operating Systems)

Lab Manual: Process Creation

Topics to be Covered

- Child Process Creation
  - fork()
- Process Invocation
  - exec() family
- Orphan and Zombie Processes

Objectives

- Students will be able to create child processes using fork() and tell the parent and child apart using the return value.
- Students will be able to replace the program running inside a process using the exec() family.
- Students will be able to explain, create and prevent orphan and zombie processes.

System Calls

- fork()
- getpid(), getppid()
- wait()
- exit()
- execl(), execlp(), execv(), execvp(), execle(), execve()
- sleep()

---

1. Process Basics

A **process** is a program in execution. Every process in UNIX/Linux has:

- a unique **Process ID (PID)**,
- a **Parent Process ID (PPID)**, the PID of the process that created it,
- its own address space (code, data, heap, stack), and
- an entry in the kernel's process table.

Every process except the very first one init is created by another process. This creates a tree of processes: the creator is the **parent** and the new process is the **child**.

Getting process IDs

Syntax

```
#include <unistd.h>

getpid();   // returns PID of the calling process
getppid();  // returns PID of the parent of the calling process
```

Sample Code 1

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("My PID  : %d\n", getpid());
    printf("My PPID : %d\n", getppid());
    return 0;
}
```

Run the program two or three times. Does the PID change? Does the PPID change? Which program is the parent?

---

2. Child Process Creation: fork()

`fork()` creates a new process by **duplicating the calling process**. The new process is the child and the original is the parent. After the call, the child gets a copy of the parent's code, data, heap, stack and open file descriptors, and **both processes continue executing from the instruction right after fork()**.

The only way to tell the two apart is the **return value** of fork():

Return value	Meaning
Negative (-1)	fork failed, no child was created. errno is set.
0	Returned **in the child** process.
Positive	Returned **in the parent**. The value is the PID of the new child.

Important points:

- fork() is called once but returns twice, once in each process.
- Parent and child have **separate copies** of variables. A change made by one is not seen by the other.
- The order in which parent and child run is decided by the scheduler. It is **not predictable**, so never depend on it.

- File descriptors are inherited. This is why a pipe created before fork() can be shared between parent and child.

Syntax

```
#include <sys/types.h>
#include <unistd.h>

pid_t pid=fork();
```

Sample Code 2: Before and after fork

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Before fork: PID = %d\n", getpid());
    fork();
    printf("After fork : PID = %d\n", getpid());
    return 0;
}
```

Sample Code 3: Separating parent and child

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    int x = 10;
    pid_t pid = fork();

    if (pid < 0)                        /* fork failed */
    {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0)                  /* child */
    {
        x = x + 5;
        printf("Child : PID=%d, PPID=%d, x=%d\n", getpid(), getppid(), x);
    }
    else                                /* parent */
    {
```

```
        x = x - 5;
        printf("Parent: PID=%d, child PID=%d, x=%d\n", getpid(), pid, x);
    }
    return 0;
}
```

Observe that the value of x is different in the two processes, because each has its own copy.

Sample Code 4: Multiple fork() calls

Every fork() doubles the number of processes. After n consecutive fork() calls there are 2^n processes in total (1 original and 2^n - 1 children).

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    fork();
    fork();
    printf("Hello from PID %d\n", getpid());
    return 0;
}
```

---

3. Waiting for a Child: wait()

When the parent must know that a child has finished (or needs its exit status), it calls `wait()`. The parent is **blocked** until one of its children terminates. wait() also lets the kernel remove the finished child from the process table (this is called *reaping* the child, and it is the key to avoiding zombies).

- `status` receives information about how the child ended. Pass NULL if it is not needed.
- wait() returns the PID of the terminated child, or -1 on error (for example, if there are no children).

Macros to examine `status`:

Macro	Meaning
WIFEXITED(status)	True if the child ended normally (exit() or return).
WEXITSTATUS(status)	The exit code of the child (lowest 8 bits, so 0 to 255).
WIFSIGNALED(status)	True if the child was killed by a signal.

Sample Code 5: Parent waits for child

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    int status;
    pid_t pid = fork();

    if (pid == 0)                       /* child */
    {
        printf("Child : working...\n");
        sleep(2);
        printf("Child : done.\n");
        exit(7);
    }
    else                                /* parent */
    {
        wait(&status);
        if (WIFEXITED(status))
            printf("Parent: child exited with code %d\n",
WEXITSTATUS(status));
    }
    return 0;
}
```

---

4. Process Invocation: the exec() Family

fork() creates a new process, but it runs the **same program** as the parent. To run a **different program**, a process calls one of the exec functions. exec **replaces** the current process image (code, data, heap, stack) with a new program loaded from disk.

Important points:

- exec does **not** create a new process. The PID stays the same.
- If exec succeeds, it **never returns**, because the old program no longer exists. Any code after exec runs only if exec **failed**.
- Open file descriptors stay open (unless marked close-on-exec).
- The usual pattern to launch a program is **fork() followed by exec() in the child**, while the parent continues or waits. This is exactly how a shell runs your commands.

Syntax

```
#include <unistd.h>

int execl (const char *path, const char *arg0, ..., NULL);
int execlp(const char *file, const char *arg0, ..., NULL);
int execle(const char *path, const char *arg0, ..., NULL, char *const envp[]);
```

```
int execv (const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execve(const char *path, char *const argv[], char *const envp[]);
```
The letters after "exec" tell you how it behaves:

Letter	Meaning
l	Arguments are passed as a **list** (arg0, arg1, ..., NULL).
v	Arguments are passed as a **vector** (array of strings ending with NULL).
p	The program is searched for in the directories of the **PATH** variable, so a full path is not needed.
e	The **environment** is passed explicitly in `envp`.

Rules to remember:

- By convention, `arg0` (or `argv[0]`) is the name of the program itself.
- The argument list must always end with `NULL`.
- **On success:** exec never returns. **On failure:** it returns -1 and sets errno (for example ENOENT if the program is not found, EACCES if permission is denied).

Sample Code 6: execl()

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Before exec: PID = %d\n", getpid());

    execl("/bin/ls", "ls", "-l", NULL);

    /* reached only if execl fails */
    perror("execl failed");
    return 1;
}
```

Sample Code 7: execvp()

```
#include <stdio.h>
#include <unistd.h>

int main()
{
```

```
    char *args[] = {"ls", "-l", "-a", NULL};

    execvp("ls", args);               /* ls is found using PATH */

    perror("execvp failed");
    return 1;
}
```

Sample Code 8: fork() + exec() together

The parent creates a child, the child turns itself into the `date` program, and the parent waits for it.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0)                /* child */
    {
        printf("Child (PID %d) is now running date\n", getpid());
        execlp("date", "date", NULL);
        perror("exec failed");        /* only if exec fails */
    }
    else                              /* parent */
    {
        wait(NULL);
        printf("Parent (PID %d): child has finished\n", getpid());
    }
    return 0;
}
```

---

5. Orphan Processes

An **orphan process** is a child process that is still running after its **parent has terminated**. A process cannot be left without a parent, so the kernel **adopts** the orphan: its new parent becomes the init process (PID 1). The orphan keeps running normally, and when it finishes, its adoptive parent reaps it, so orphans are harmless.

How to recognise one: the value of `getppid()` inside the child **changes** after the original parent exits.

Sample Code 9: Creating an orphan

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0)                /* child */
    {
        printf("Child : PID=%d, parent PID=%d\n", getpid(), getppid());
        sleep(5);                     /* parent exits during this time */
        printf("Child : PID=%d, parent PID=%d (after parent died)\n",
               getpid(), getppid());
    }
    else                              /* parent */
    {
        printf("Parent: PID=%d exiting now\n", getpid());
        exit(0);            /* parent ends without wait() */
    }
    return 0;
}
```

---

6. Zombie Processes

When a child terminates, the kernel frees its memory but **keeps a small entry in the process table** (PID, exit status, resource usage) so that the parent can read the exit status later. This is done through wait()/waitpid().

A **zombie process** is a child that has **terminated, but whose parent has not yet called wait()**. It uses no CPU and almost no memory, but it still occupies a slot in the process table. A zombie cannot be killed with `kill`, because it is already dead. It disappears only when:

- the parent calls wait()
- the parent itself terminates (the zombie is adopted by init, which reaps it immediately).

Comparison

	Orphan	Zombie
Who has ended?	The **parent**	The **child**
Is it still running?	Yes	No (already terminated)
Shown by `ps` as	Normal state (R or S)	State **Z**, marked `<defunct>`
Problem?	No, it is adopted by init	Wastes a process-table entry until reaped
Fix	Nothing needed	Parent must call wait()

Sample Code 10: Creating a zombie

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0)                /* child */
    {
        printf("Child : PID=%d exiting immediately\n", getpid());
        exit(1);
    }
    else                              /* parent */
    {
        printf("Parent: PID=%d sleeping for 30 seconds without wait()\n",
getpid());
        sleep(30);                    /* child stays a zombie during this
time */
        printf("Parent: exiting\n");
    }
    return 0;
}
```

Steps to observe the zombie

1. Compile and run the program in one terminal.

2. Open a second terminal and run: `ps -l`
3. Find the child's line. Its state column shows **Z** and its name is followed by `<defunct>`.
4. Wait for the parent to exit and run `ps` again. The zombie is gone.

Sample Code 11: Preventing the zombie

The fix is to make the parent call wait() so that the child is reaped as soon as it ends.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0)                /* child */
    {
        printf("Child : PID=%d exiting immediately\n", getpid());
    }
    else                              /* parent */
    {
        wait(NULL);                   /* ...until it is reaped here */
        printf("Parent: child reaped, no zombie remains\n");
    }
    return 0;
}
```

---

7. Quick Reference

Function	Purpose	Returns
fork()	Create a child process (copy of the caller)	Child PID in parent, 0 in child, -1 on error
getpid() / getppid()	Get own PID / parent's PID	The PID
wait()	Wait for a child to end and reap it	PID of the child, -1 on error

Function	Purpose	Returns
exec family	Replace the current program with a new one	Never returns on success, -1 on error
exit()	Terminate the process with a status code	Does not return

---

Practice Questions

Part A: Predict the Output

For each program, write down how many processes are created in total (including the original) and how many times the printf line is executed.

Question 1

```
int main()
{
    fork();
    fork();
    fork();
    printf("Hello\n");
    return 0;
}
```

Question 2

```
int main()
{
    if (fork() == 0)
        printf("A\n");
    else
        printf("B\n");
    printf("C\n");
    return 0;
}
```

Question 3

```
int main()
{
    int i;
    for (i = 0; i < 3; i++)
        fork();
    printf("X\n");
    return 0;
}
```

Question 4

```
int main()
{
    printf("Start\n");
    execl("/bin/echo", "echo", "Hello", NULL);
    printf("End\n");
    return 0;
}
```

Part B: Programming Tasks

Task 1: Parent and child IDs Write a C program in which the parent creates one child. The child prints its own PID and its parent's PID. The parent prints its own PID and the PID of the child. Make the parent wait for the child before it prints.

Task 2: Three children Write a C program in which the parent creates exactly three child processes using a loop. Each child prints its child number (1, 2 or 3) and its PID, and then exits. The parent must wait for all three children and then print "All children finished". Make sure the children do not create further children.

Task 3: Process chain Write a C program that creates a chain of three generations: a parent creates a child, and that child creates a grandchild. Each process prints its PID and PPID. Arrange the waits so that the output order is always: grandchild, child, parent.

Task 4: Exit status Write a C program that takes an integer n from the command line (argv). The child computes the sum of the numbers from 1 to n and returns it to the parent through its exit status. The parent retrieves it with wait() and prints it. Test with n = 5 and n = 30, and explain what happens when the sum is larger than 255.

Task 5: Running programs with exec Write a C program in which the parent creates two children. Child 1 runs `ls -l` using execvp(), and child 2 runs `date` using execlp(). The parent waits for both children and prints the exit status of each.

Task 6: Mini command runner Write a C program that takes a command and its arguments from the command line (for example `./runner ls -l /home`). The program forks, the child runs that command using execvp(), and the parent waits and then prints "Command finished". If the command does not exist, the child must print an error using perror() and exit with a non-zero status, and the parent must report this.

Task 7: Orphan demonstration Write a C program that creates an orphan process. The child must print its PPID before and after the parent exits, so that the change is visible. Explain, in a comment in your code, which process became the new parent.

Task 8: Zombie demonstration and fix Write a C program in which the parent creates 3 children that exit immediately, and then sleeps for 20 seconds before doing anything else. While it sleeps, show the 3 zombies using `ps` in another terminal (take a screenshot). Then modify the program so that no zombie is left behind, and show the `ps` output again.